Applied Machine Learning in the Cloud
E6998

Prof. Seetharami Seelam

Prof. I-Hsin Chung

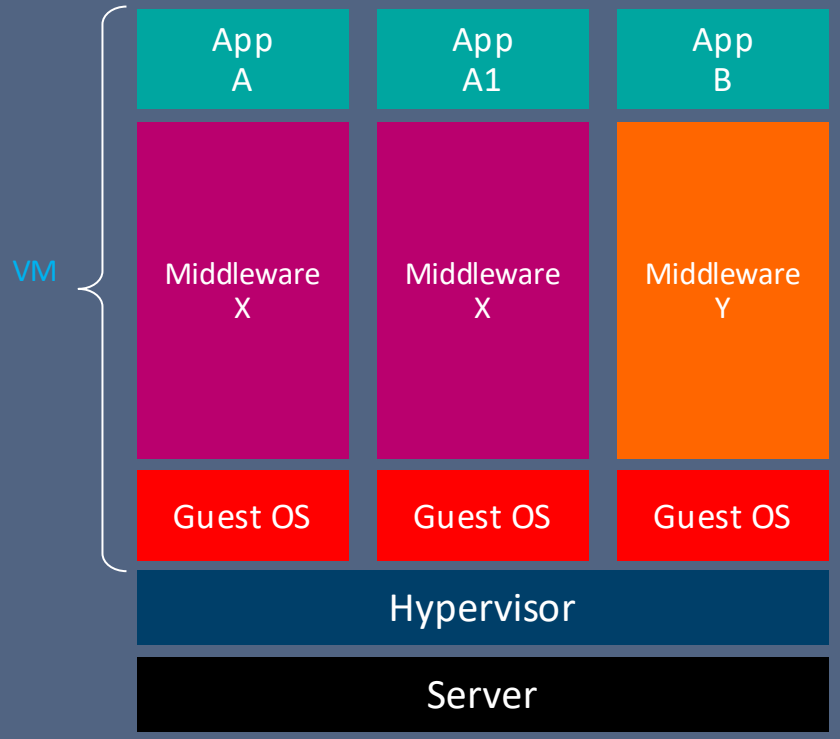
Lecture 12: Containers, Kubernetes, and Kubeflow

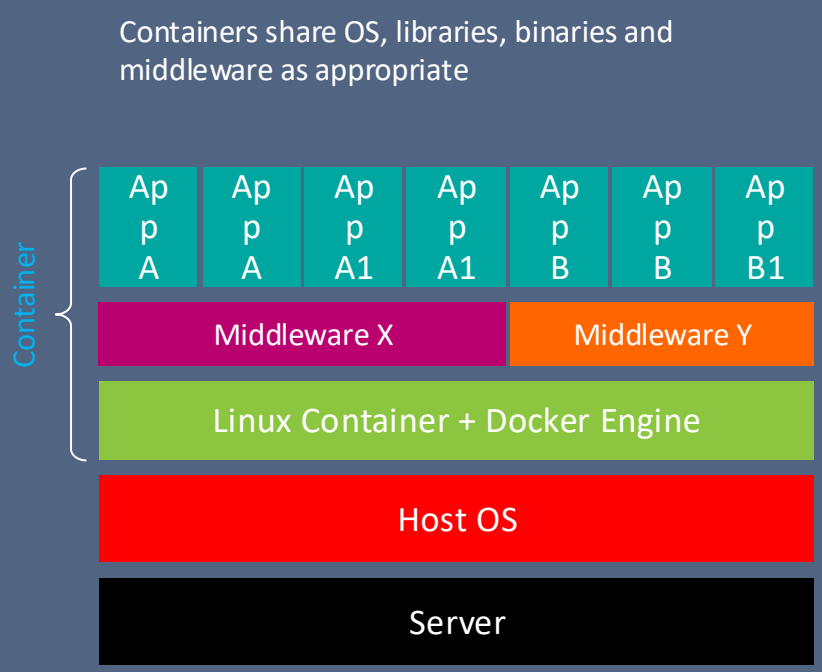
Lecture 12

- Lecture
  - Introduction to Containers
  - Docker and ecosystem
  - **Kubernetes**
  - Kubeflow – AI training
- Lab
  - Kubernetes on GCP
  - Use GKE to run an AI training job

What's a Container and how it differs from a VM?

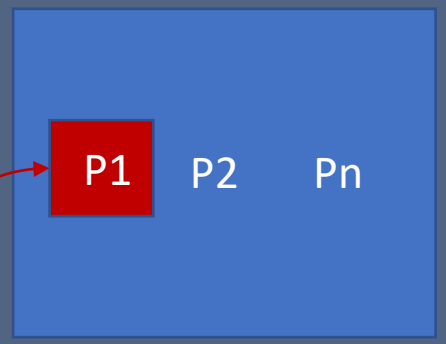
- The concept of containers emerged a decade ago (e.g. Sun Solaris Zones and IBM AIX's WPARs). **Docker** is built on open source container capabilities inside Linux kernel (cgroups, namespaces, selinux, etc)
- A container encapsulates an application and its dependencies which run in an isolated process on the host's operating system (all application share the same OS)
- Traditional hardware virtualization creates an entire virtual machine. Each VM contains not only the application (which may only be 10's of MB) but must include and an entire Guest operating System (which may measure in 1-10s of GB).





What is a container?

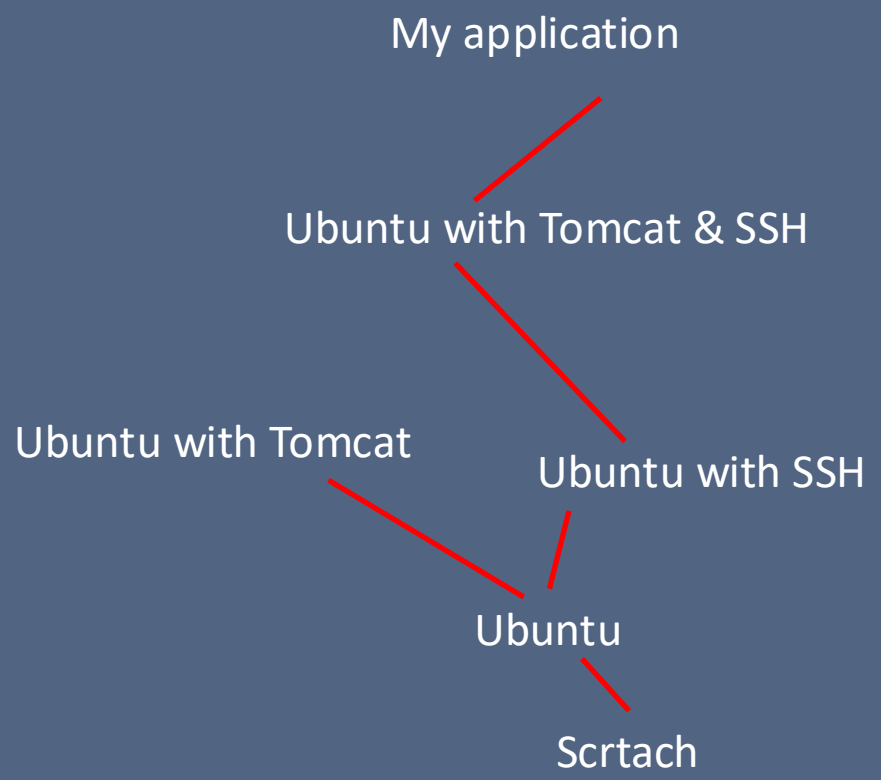
Runtime: A sandbox for a process



Name space (pid, network)
Cgroups (Memory, CPU, GPU)

Host: Linux

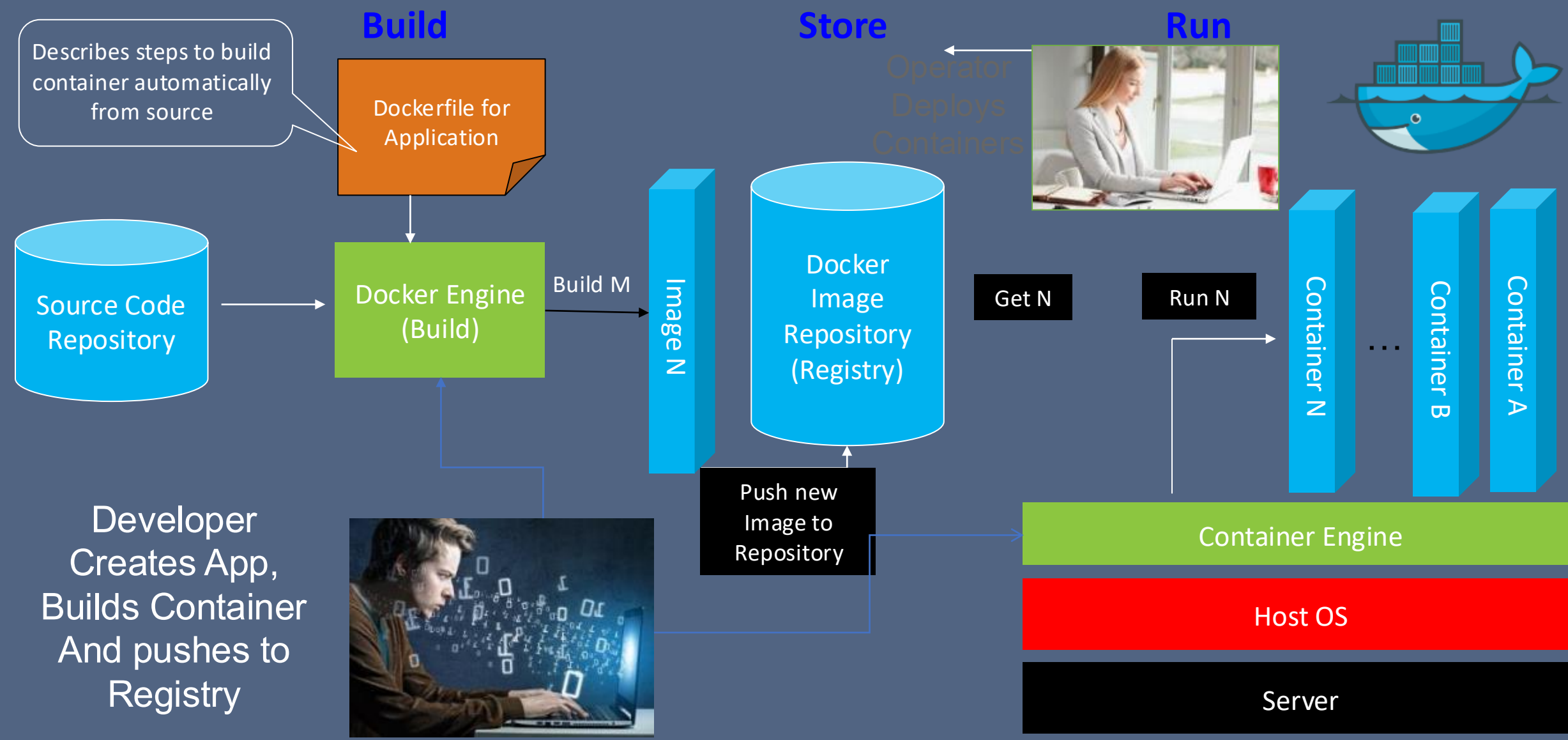
Image



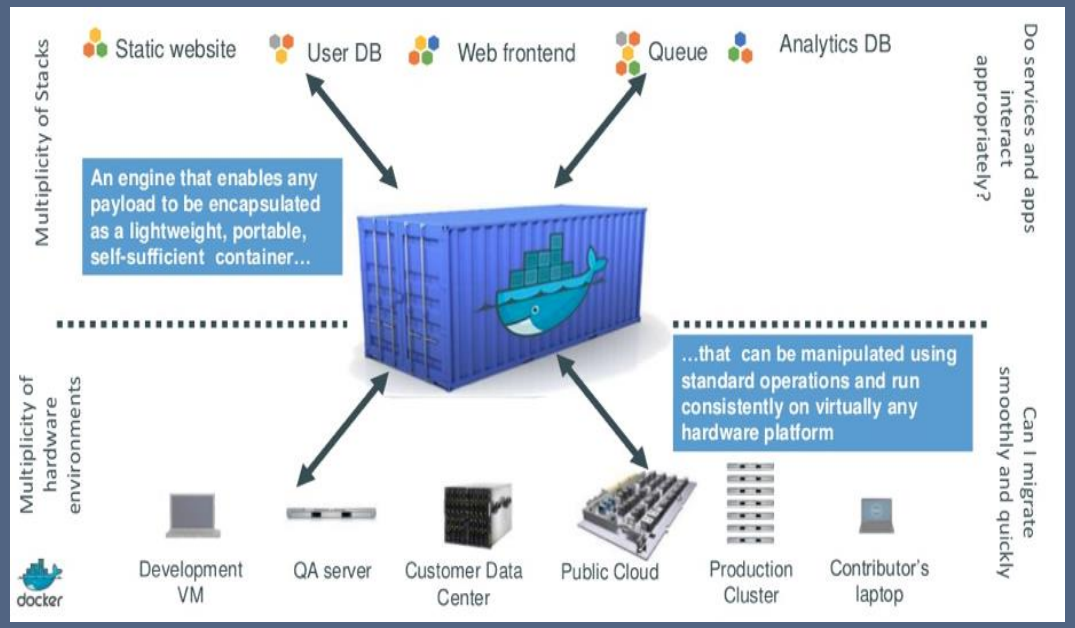
Dockerfile

```
From
....
.....
.....
```

What are the Basic Functions of Containers



Docker is an evolution of the Container Technology



Dev ⟷ Test ⟷ Production

Light weight virtualization container
- Portable
- Fast start
- Lightweight
- Tool and container ecosystem

Potential impact
- DevTest
- Platform as a Service
- Cloud Platform Services
- Software delivery

- Provision in seconds / milliseconds
- Near bare metal runtime performance
- VM-like agility – it's still "virtualization"
- Flexibility
  - Containerize "application(s)"
  - Deliver Polyglot apps
  - Repeatability
- Lightweight
- Open source – free – lower TCO
- Supported OOTB modern Linux kernel
- Runs on baremetal
- Growing set of tools and ecosystem
- Versioning and Portability
- Others: containerd, cri-o, lmctfy, rkt, wpar, Solaris zones

Containers vs VMs

Containers	VMs
Share the host operating system	Each VM has an operating system
Light weight	Heavy weight
Native performance	Some overhead
Memory and CPU can be changed flexibly	Change in Memory/CPU changes require reboot
Isolation at the process	Fully isolated
Virtually no startup time	Startup time in minutes
Support for HW like GPUs/FPGA not fully mature	Mature support for HW

Containers and VMs will co-exist for a long time

What Can you Do with containers?

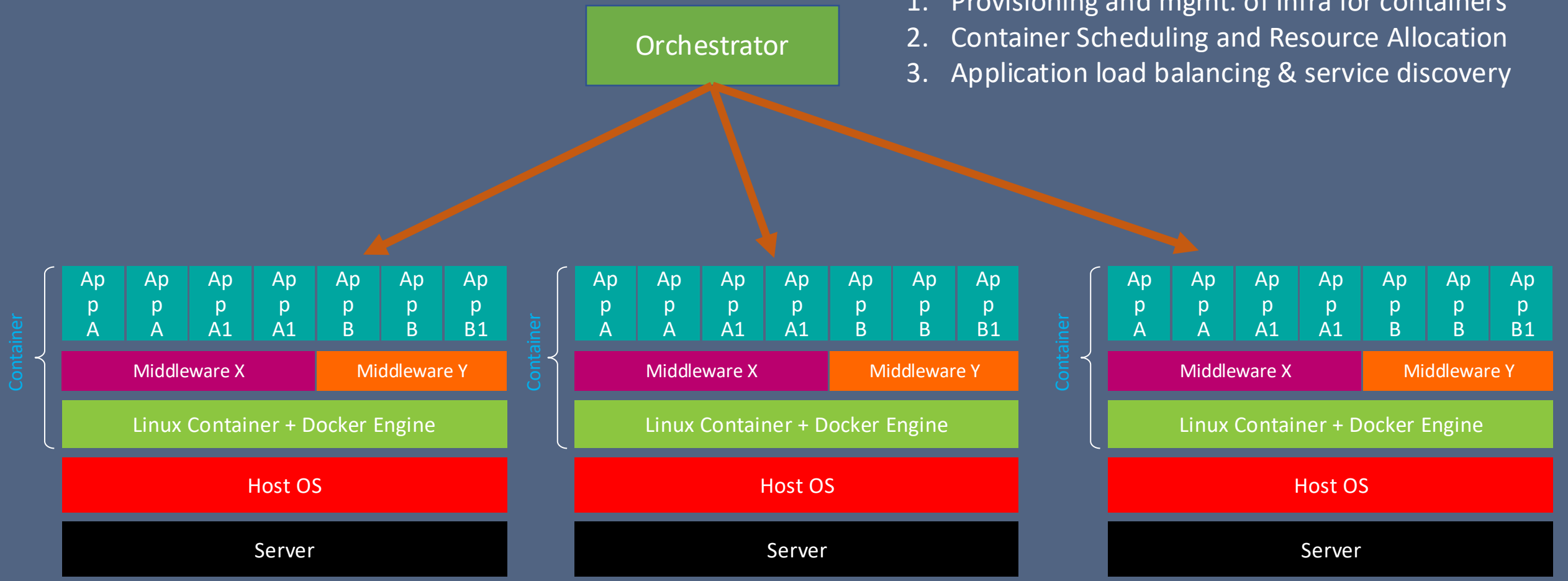
- You can run Containers from the Images in the container Registry (e.g., Docker Hub, icr)
- You can build container Images that hold your applications and their dependencies
- You can create Containers from those container images to run your applications
- You can share those container images via Docker Hub or your own container registry
- You can pull those images from the Docker registry to deploy them as Containers on a server running Docker Engine
- You can even deploy those containers in the IBM Container Cloud!

Docker ecosystem

- Docker github:
  - https://github.com/docker
- Docker registry
  - https://hub.docker.com
- Orchestration
  - Docker swarm, Kubernetes, Mesos, OpenStack, etc

What is container orchestration?

1. Provisioning and mgmt. of Infra for containers
2. Container Scheduling and Resource Allocation
3. Application load balancing & service discovery

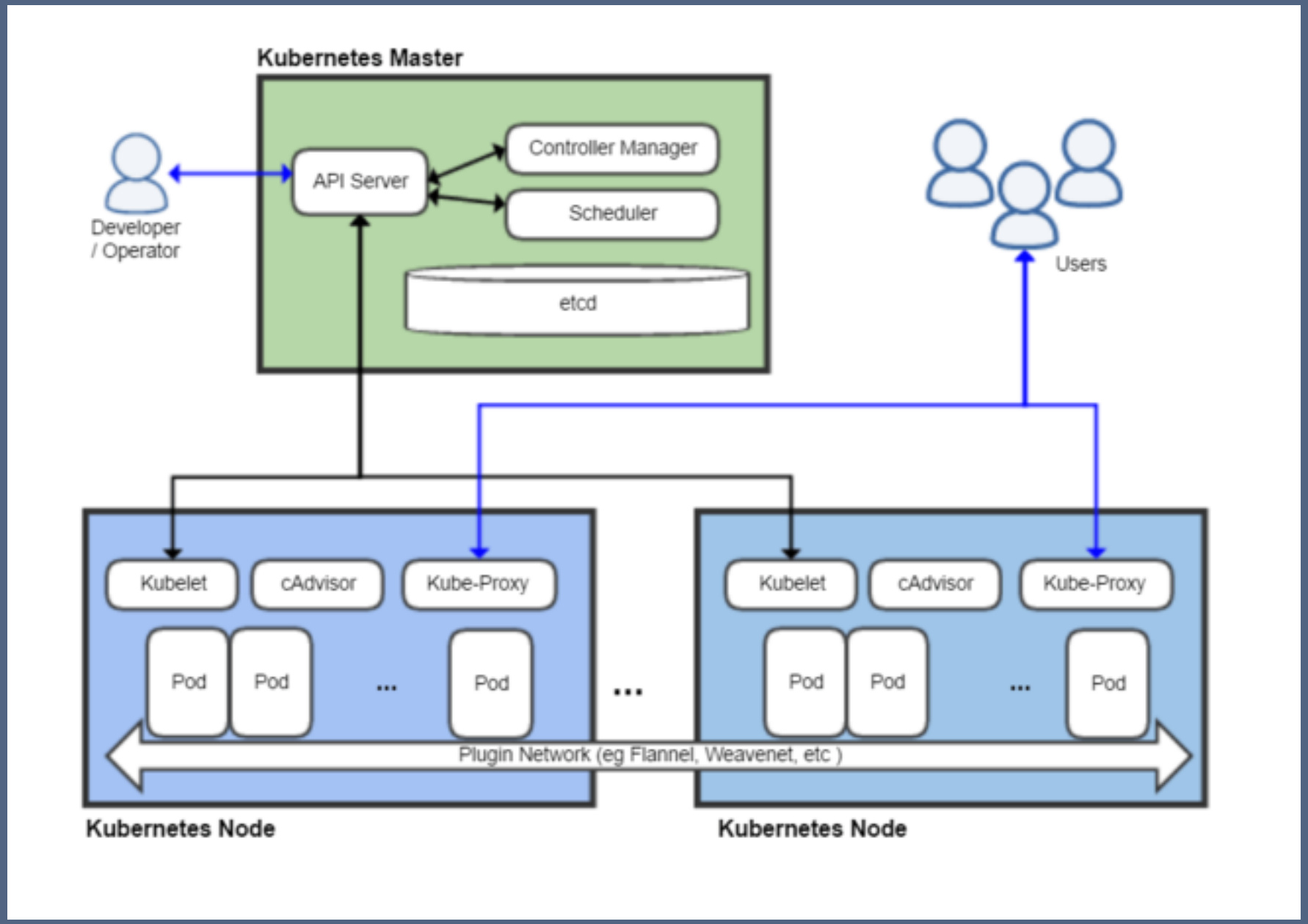


Container orchestrators

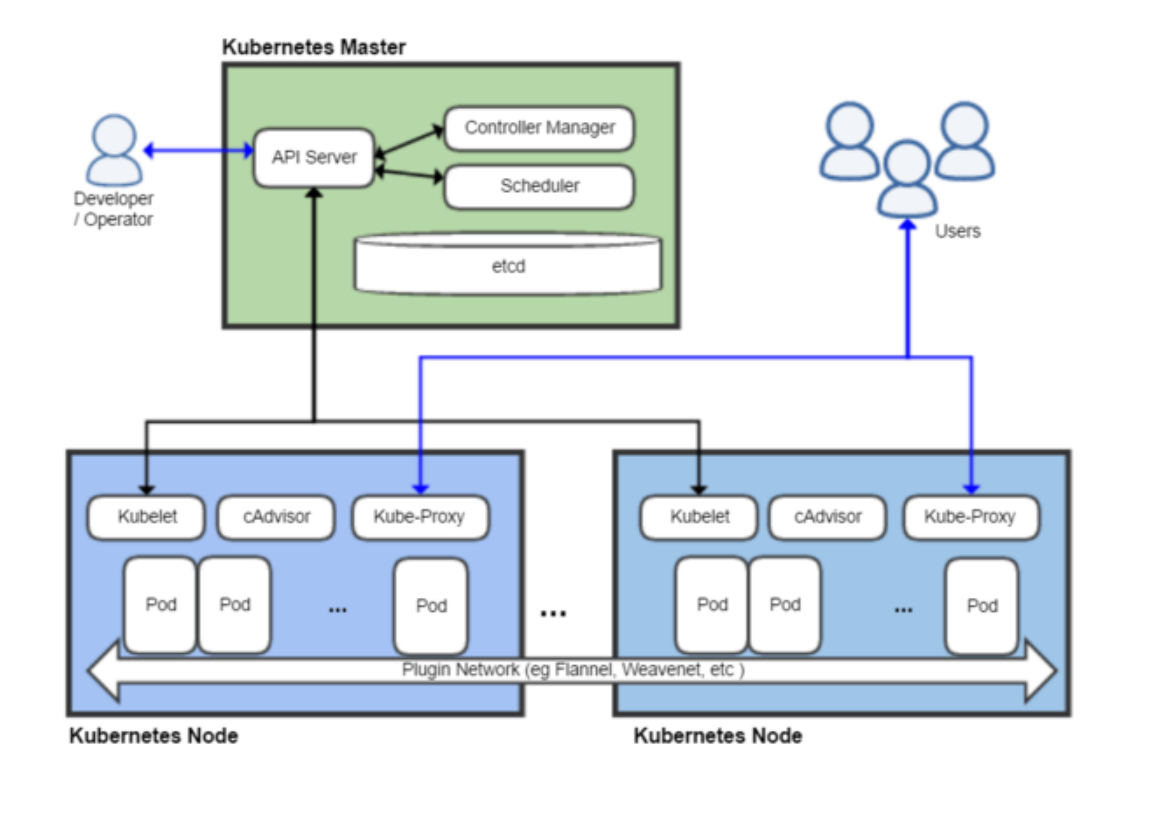
- Apache Mesos & DCOS
- Mesos Marathon
- Docker Swarm
- Docker Compose
- CoreOS Fleet
- Tupperware (twine)
- **OpenShift**
- **Kubernetes**

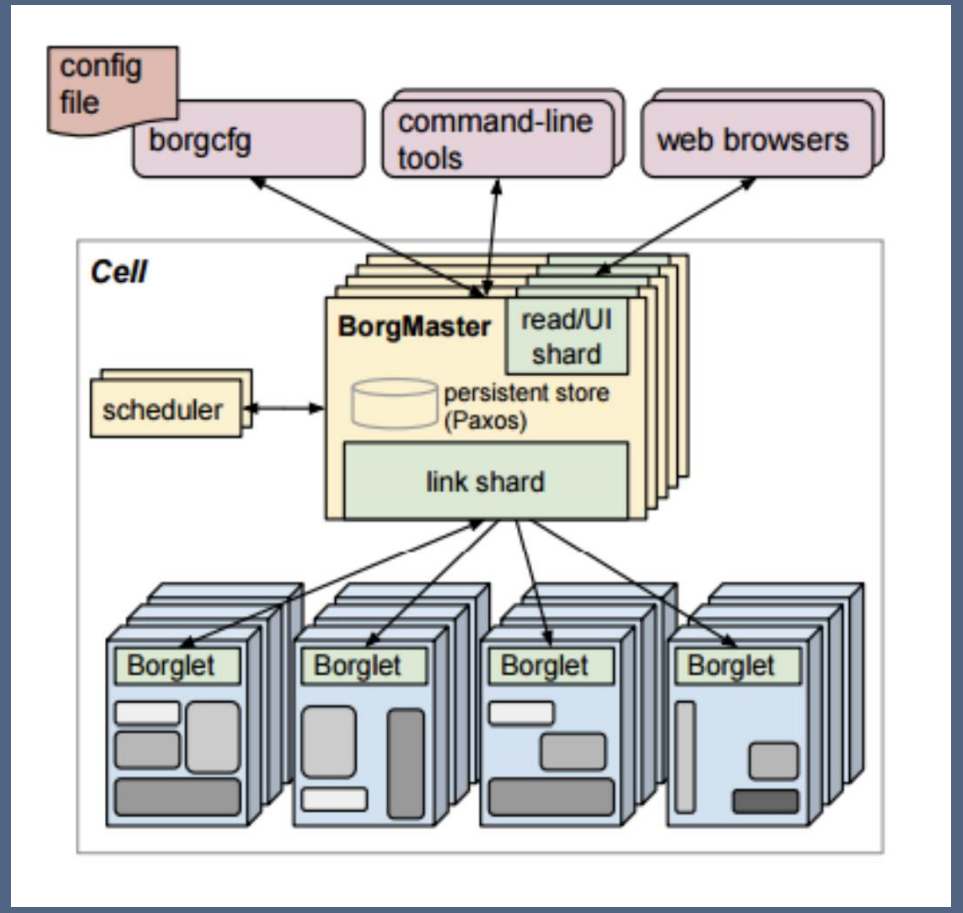
What is Kubernetes?

- “Kubernetes is a portable, extensible open-source platform for managing containerized workloads and services, that facilitates both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.”
  - From: https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/


Kubernetes Master
Developer
/ Operator
API Server
Controller Manager
Scheduler
etcd
Users
Kubelet
cAdvisor
Kube-Proxy
Pod
Pod
...
Pod
...
Kubelet
cAdvisor
Kube-Proxy
Pod
Pod
...
Pod
Plugin Network (eg Flannel, Weavenet, etc)
Kubernetes Node
Kubernetes Node

Kubernetes vs Borg

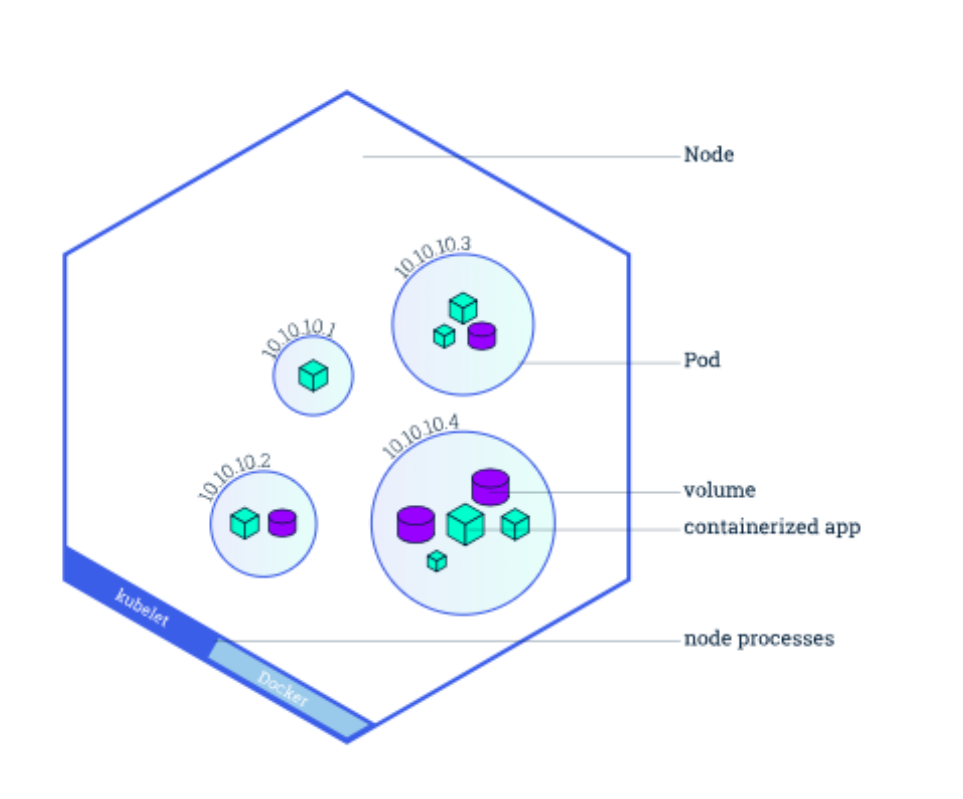




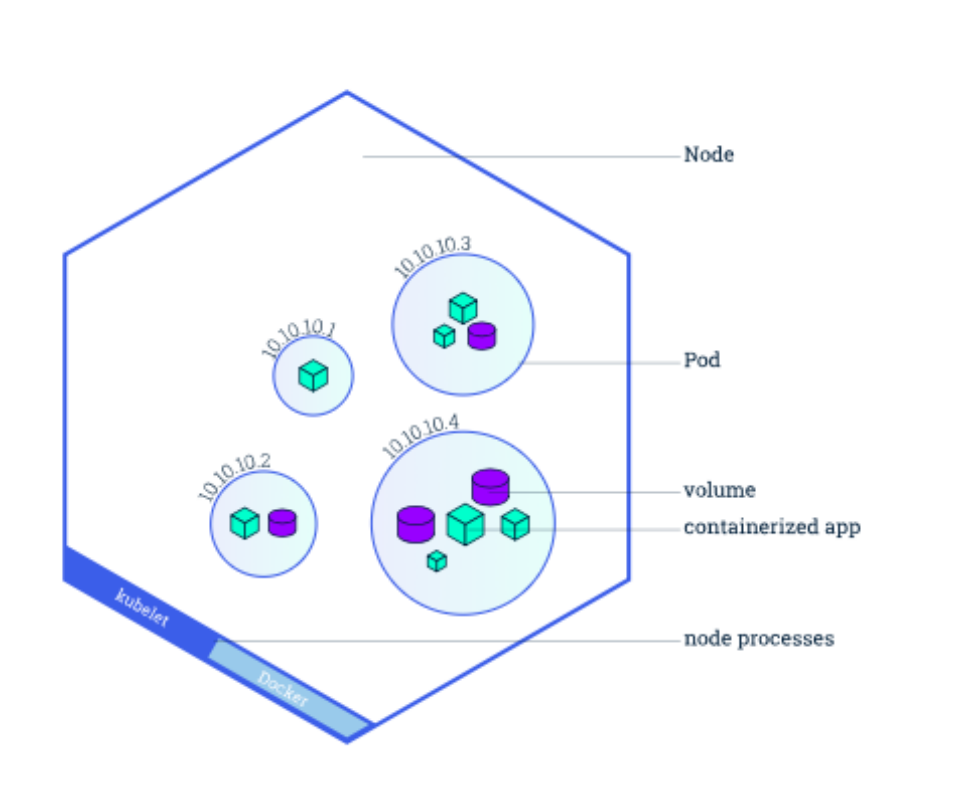
Kubernetes basic objects and controllers

- Basic Kubernetes objects
  - **Pod**
  - **Service**
  - **Volume**
  - Namespace
- Controllers
  - **ReplicaSet**
  - **Deployment**
  - StatefulSet
  - DaemonSet
  - Job
- Kubernetes Machinery
  - **Kube API Server**
  - **Kube Scheduler**
  - **Controller-manager**
  - **Kubelet**
  - **Kube-proxy**

Nodes & Pods


Node
10.10.10.3
10.10.10.1
Pod
10.10.10.4
10.10.10.2
volume
containerized app
kubelet
Docker
node processes

Nodes & Pods

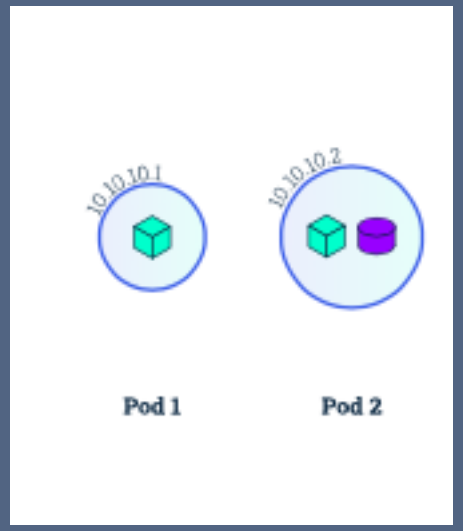


Node:

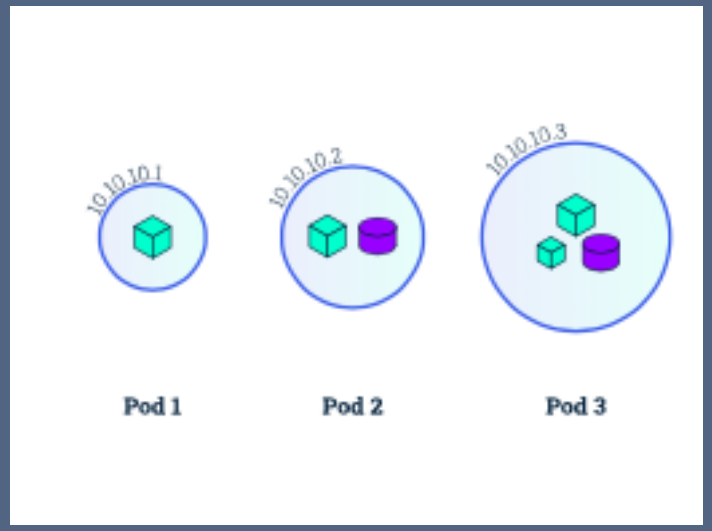
1. Kubelet (communication between master → Node)
2. Container runtime (docker, cri-o, podman)

Nodes & Pods

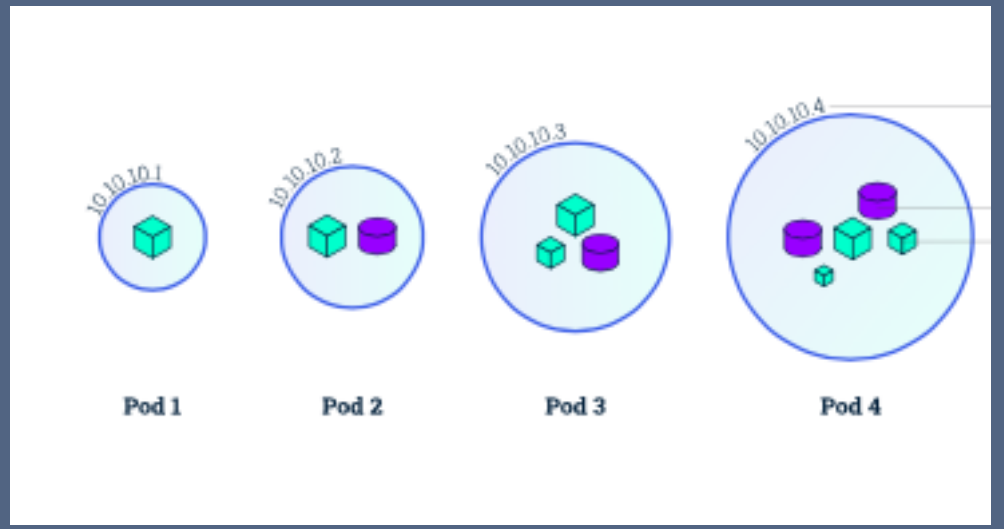
Nodes & Pods



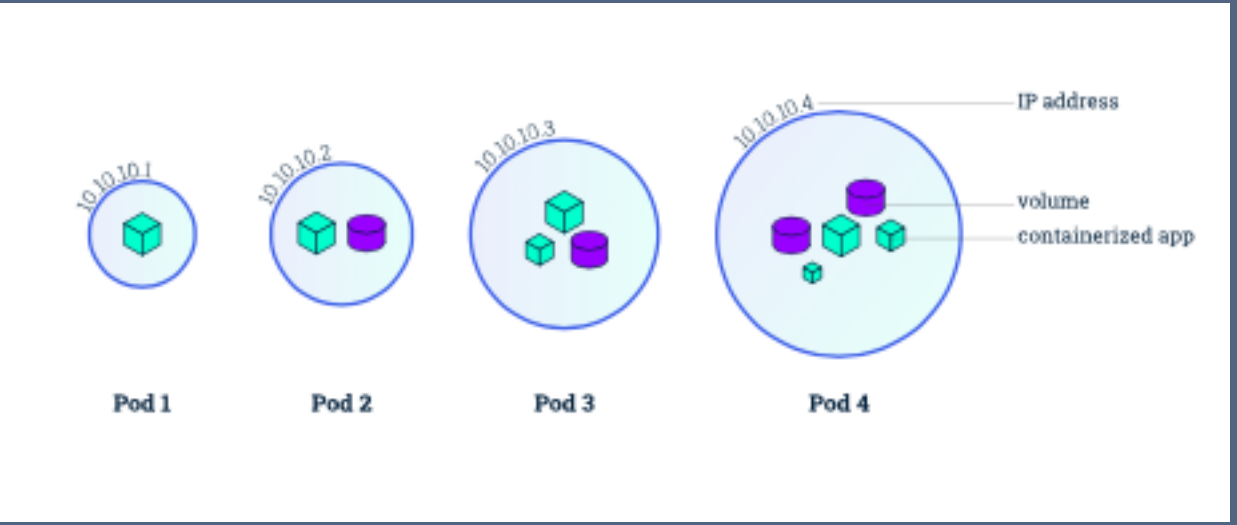
Nodes & Pods



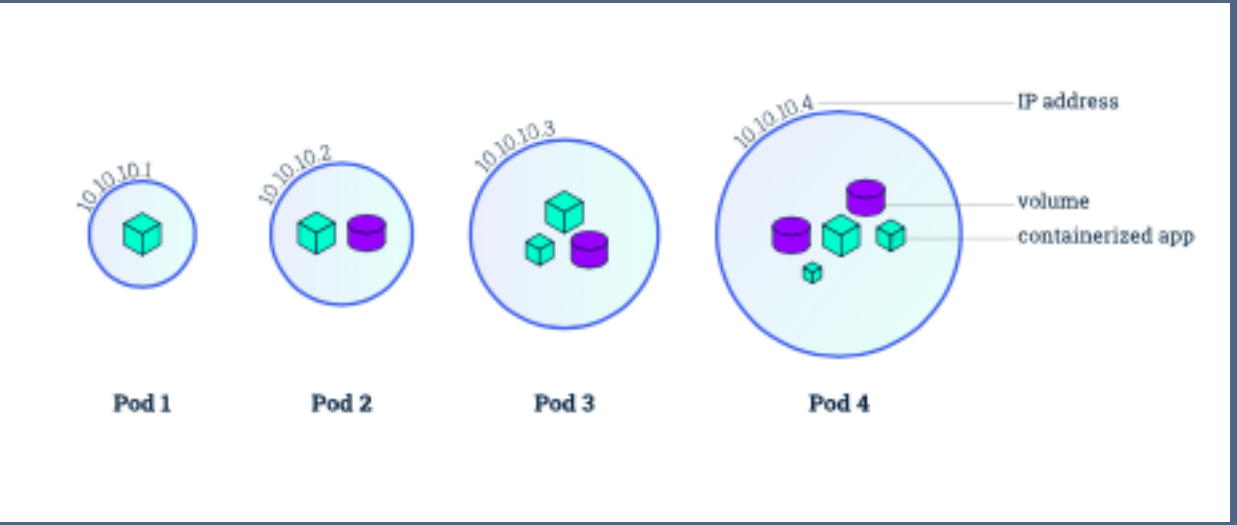
Nodes & Pods



Nodes & Pods


10.10.10.1
10.10.10.2
10.10.10.3
10.10.10.4
IP address
volume
containerized app
Pod 1
Pod 2
Pod 3
Pod 4

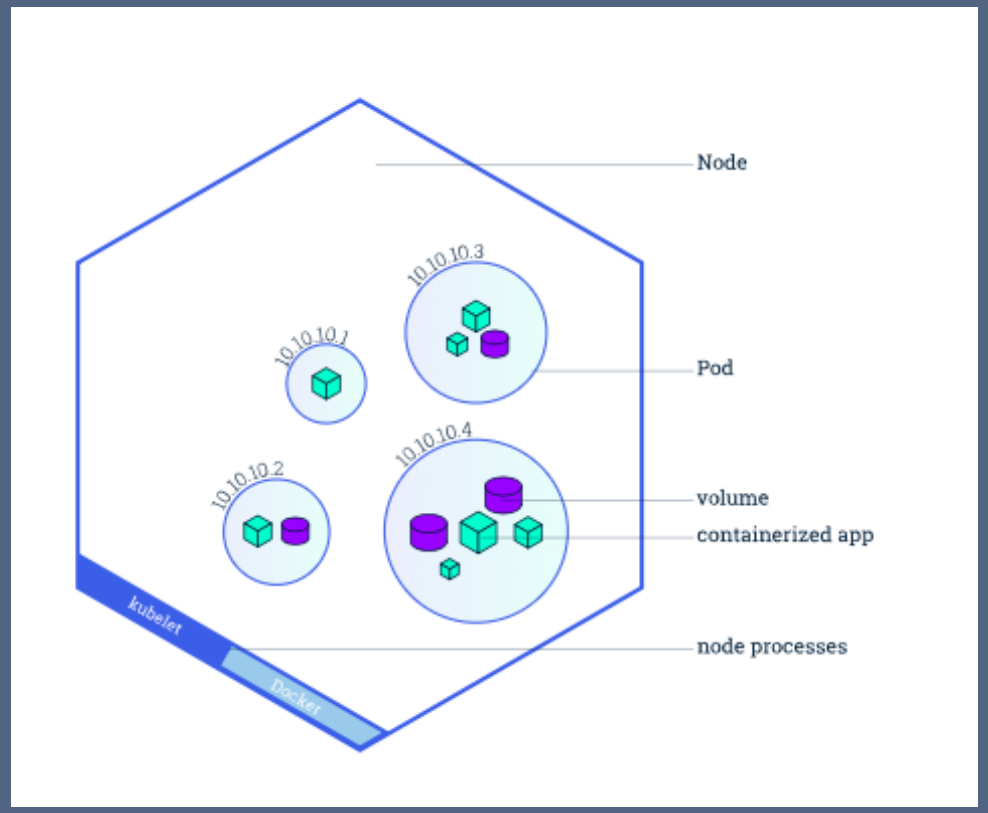
Nodes & Pods

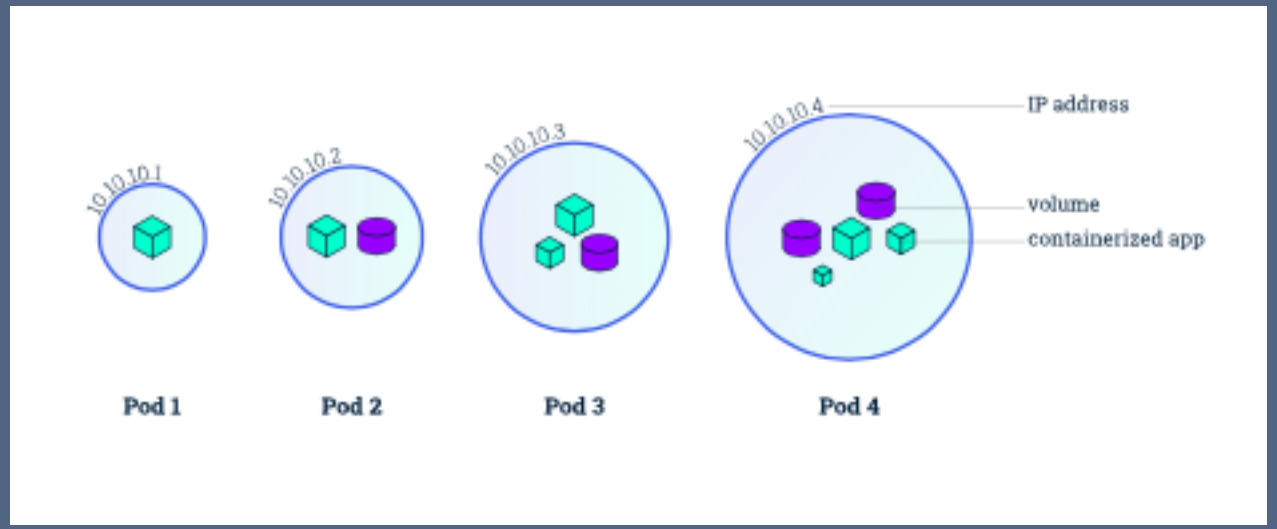


Pod:

1. Containers run in Pods
2. Shared storage
3. Shared network
4. Container behavior (ports, health, resources)
5. All containers of a pod run on the same machine

Nodes & Pods





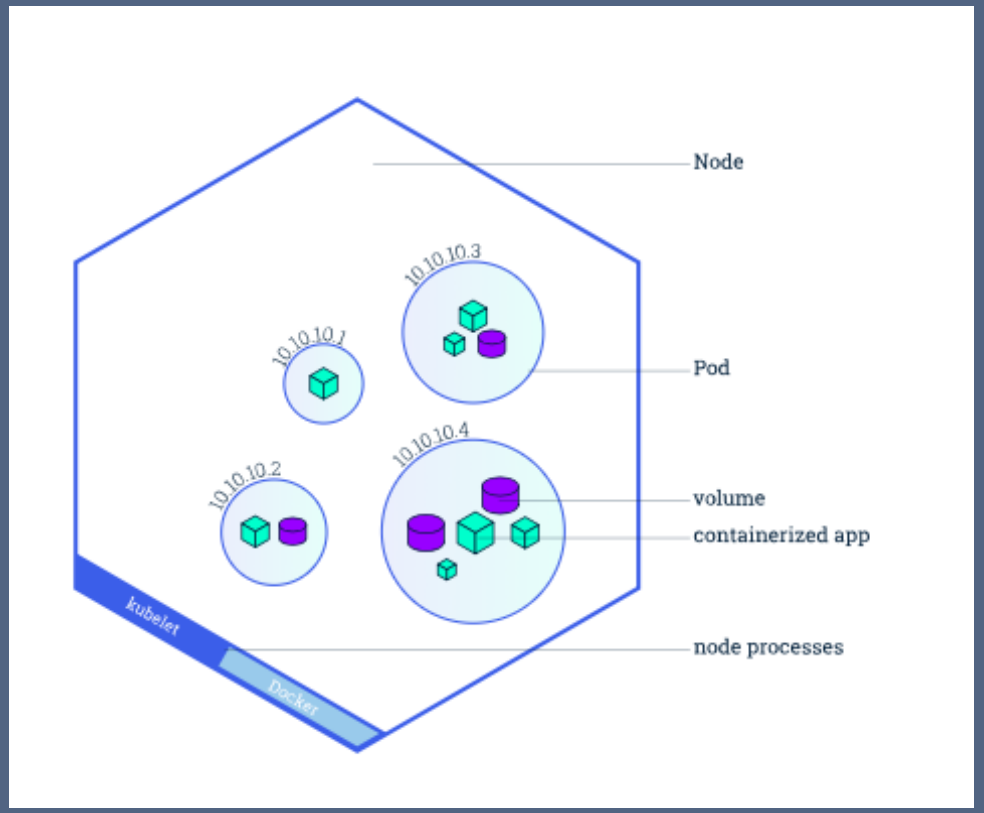
Pod:

1. Containers run in Pods
2. Shared storage
3. Shared network
4. Container behavior (ports, health, resources)
5. All containers of a pod run on the same machine

Node:

1. Kubelet (communication between master → Node)
2. Container runtime (docker, rkt, runc)

K8S Pod vs real PODS



“Hello world” pod spec

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: busybox
    command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']
```

```
-bash-4.2$ kubectl create -f hello.yaml
pod/myapp-pod created
-bash-4.2$ kubectl get pods
NAME                                          READY     STATUS              RESTARTS   AGE
ibm-cert-manager-cert-manager-768b66977-qp6db 1/1       Running             0          12d
myapp-pod                                     0/1       ContainerCreating   0          5s
test-pd                                       1/1       Running             0          2d10h
-bash-4.2$
```

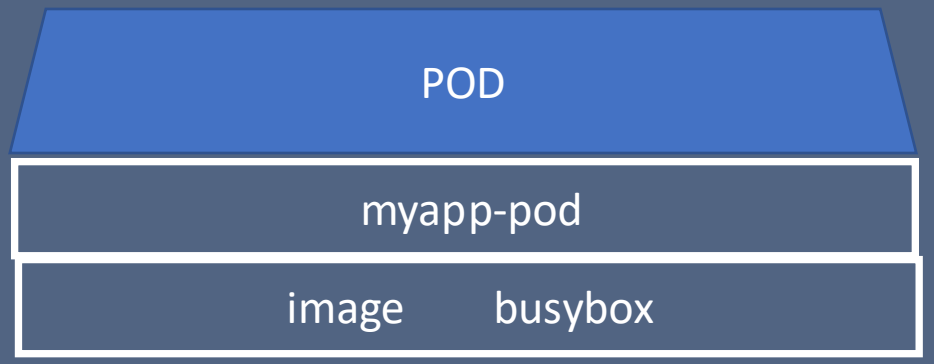
Differences between Docker and K8S

```
docker run busybox /bin/sh -c 'echo hello && sleep 10'
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: busybox
    command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']
```

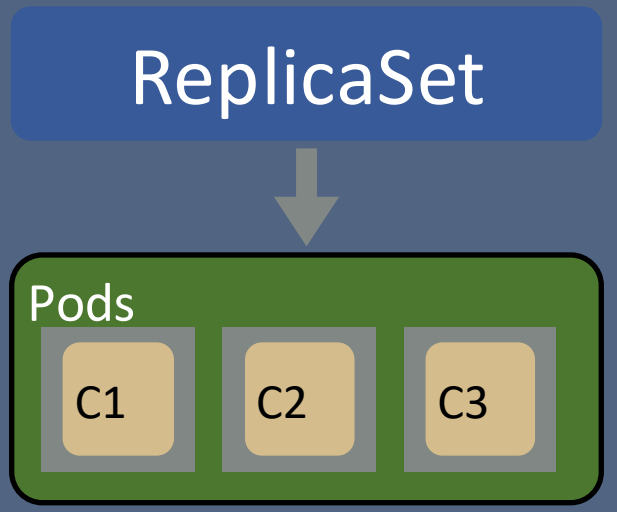
Pod spec to Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: busybox
    command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']
```



Kubernetes ReplicaSets

- ReplicaSets allow multiple copies of pods to be deployed
- One or more containers in a pod
- If a container dies, the ReplicaSet will spawn a new one



ReplicaSet

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-rs
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      tier: myapp-rs
  template:
    metadata:
      labels:
        tier: myapp-rs
    spec:
        containers:
        - name: myapp-container
          image: busybox
          command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']
```

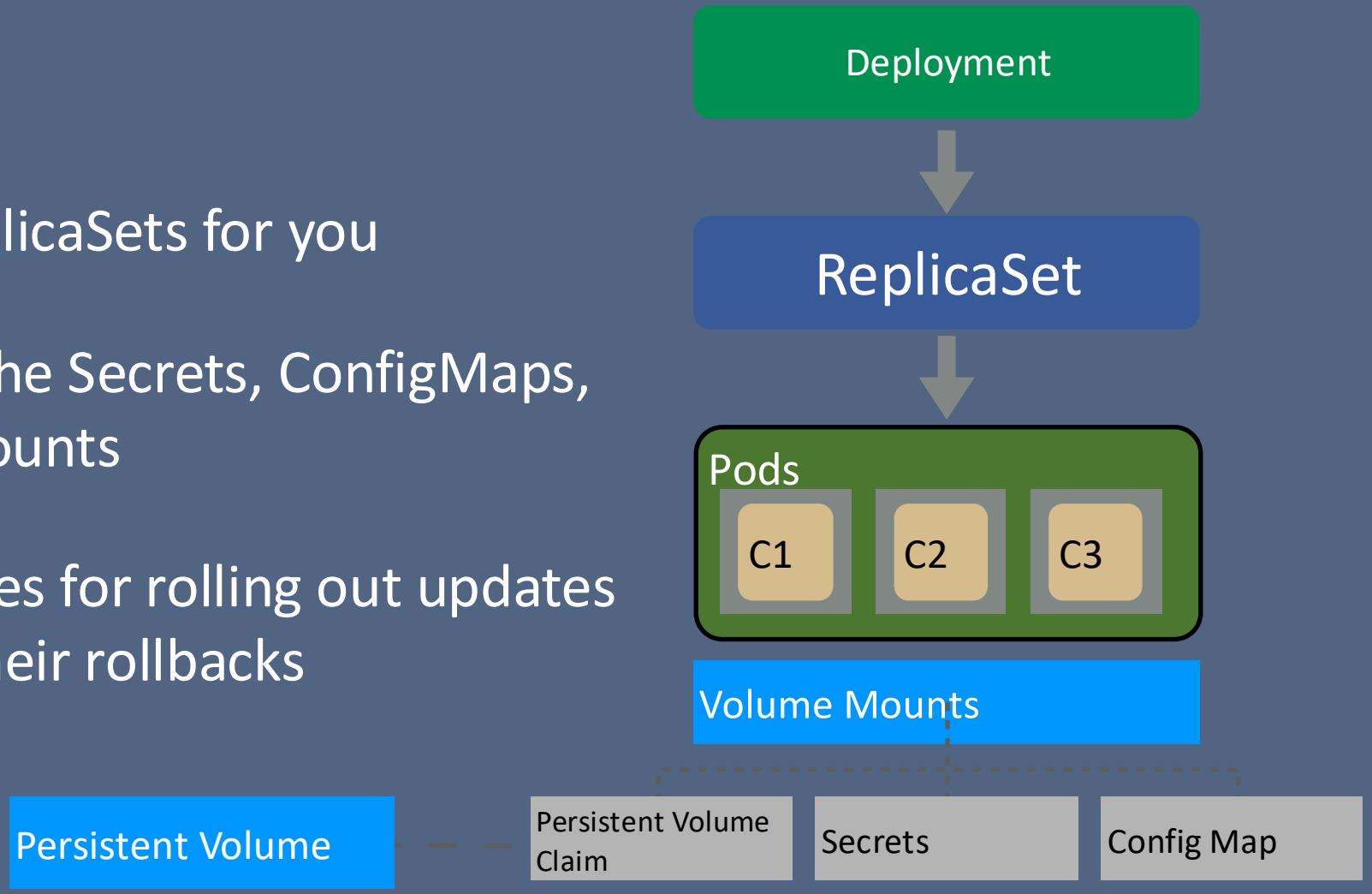
ReplicaSet vs Pod

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-rs
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      tier: myapp-rs
  template:
    metadata:
      labels:
        tier: myapp-rs
    spec:
        containers:
        - name: myapp-container
          image: busybox
          command: ['sh', '-c', 'echo Hello Kubernetes
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: busybox
    command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 36']
```

Kubernetes Deployments

- Deployment
  - Sets up the ReplicaSets for you
  - Also specified the Secrets, ConfigMaps, and Volume Mounts
  - Provides features for rolling out updates and handling their rollbacks

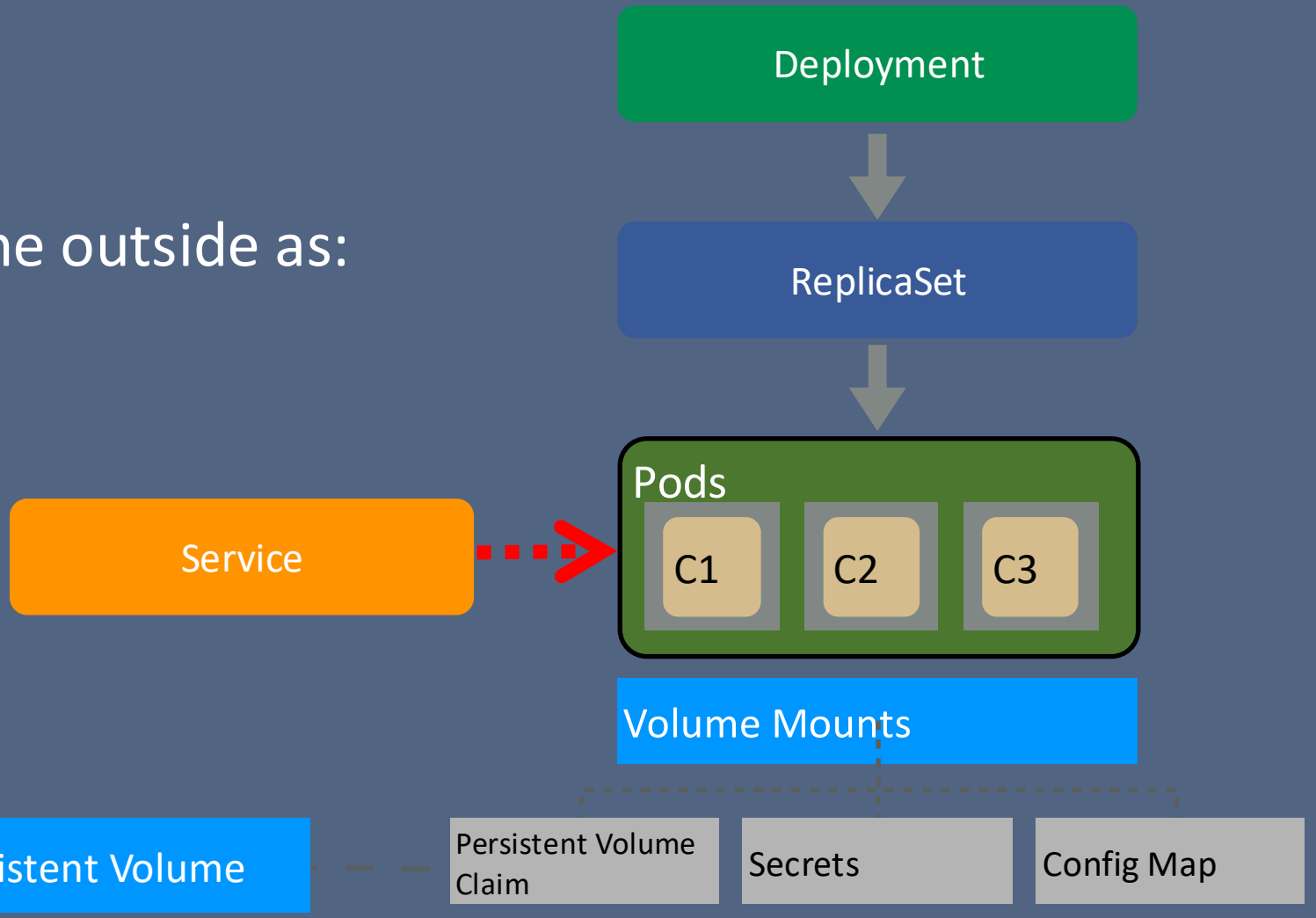


Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deployment
  labels:
    app: hello-d
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello-d
  template:
    metadata:
      labels:
        app: hello-d
    spec:
    spec:
        containers:
        - name: myapp-container
          image: busybox
          command: ['sh', '-c', 'echo Hello Kubernetes! I am updated!! && sleep 3600']
```

Kubernetes Service

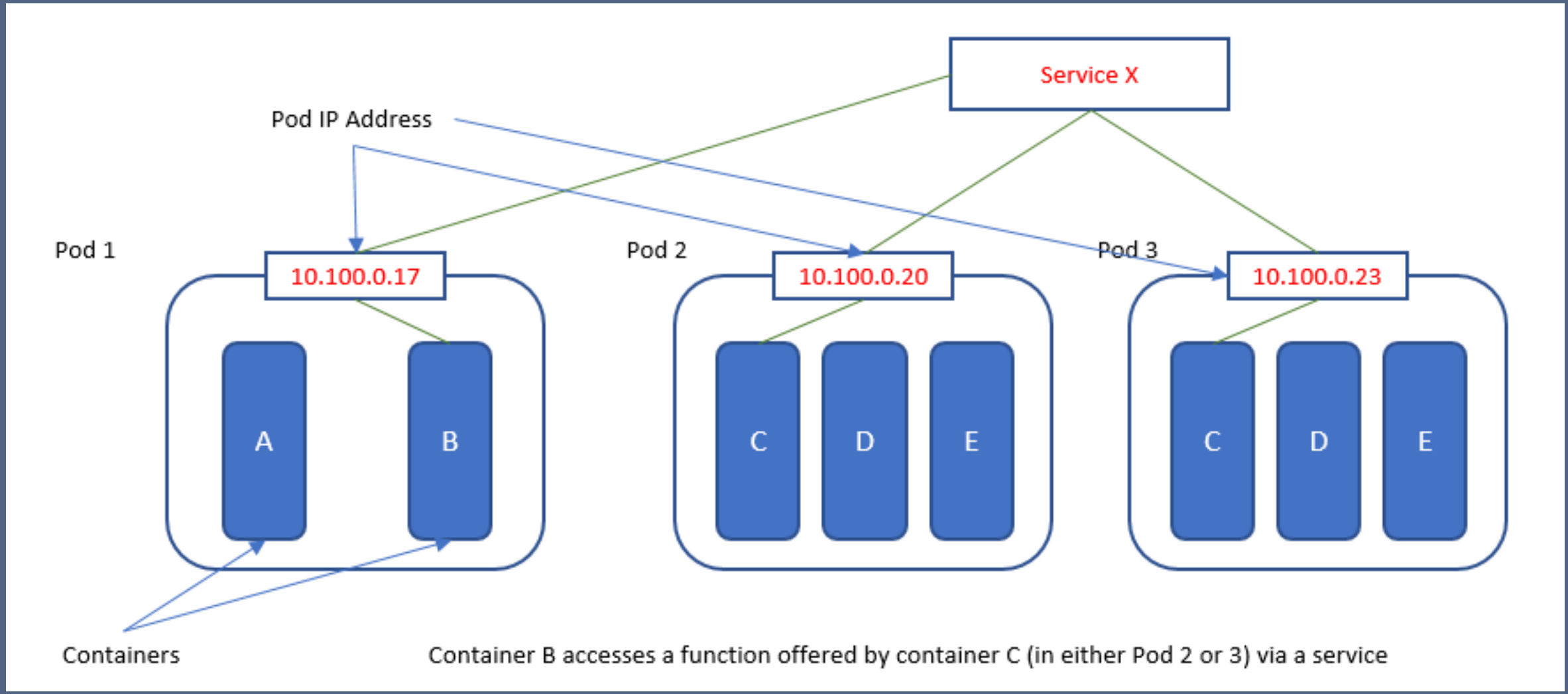
- Service
  - Exposes Pods to the outside as:
    - ClusterIP
    - NodePort
    - Load Balancer



Service Example

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: hello-d
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

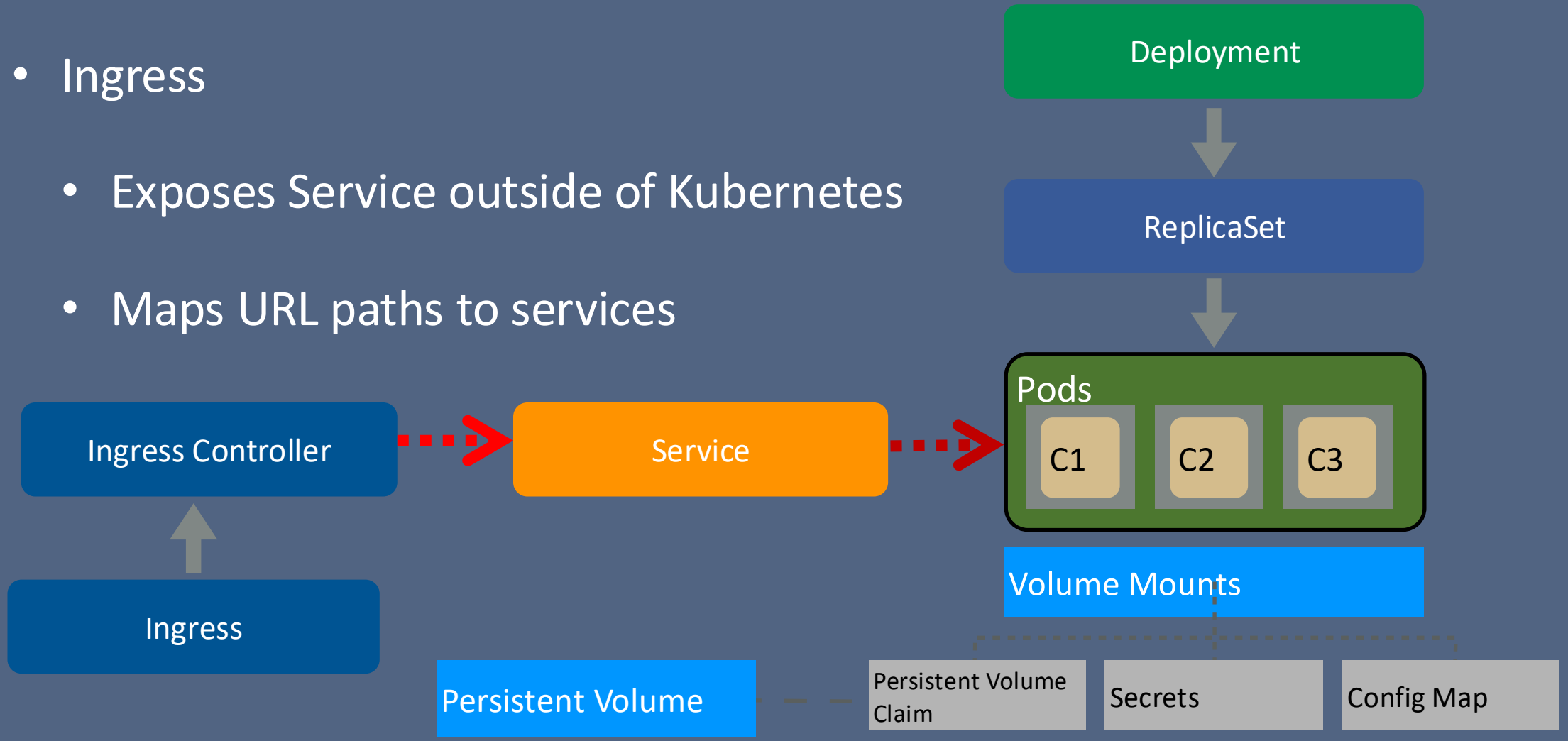
Kubernetes Service



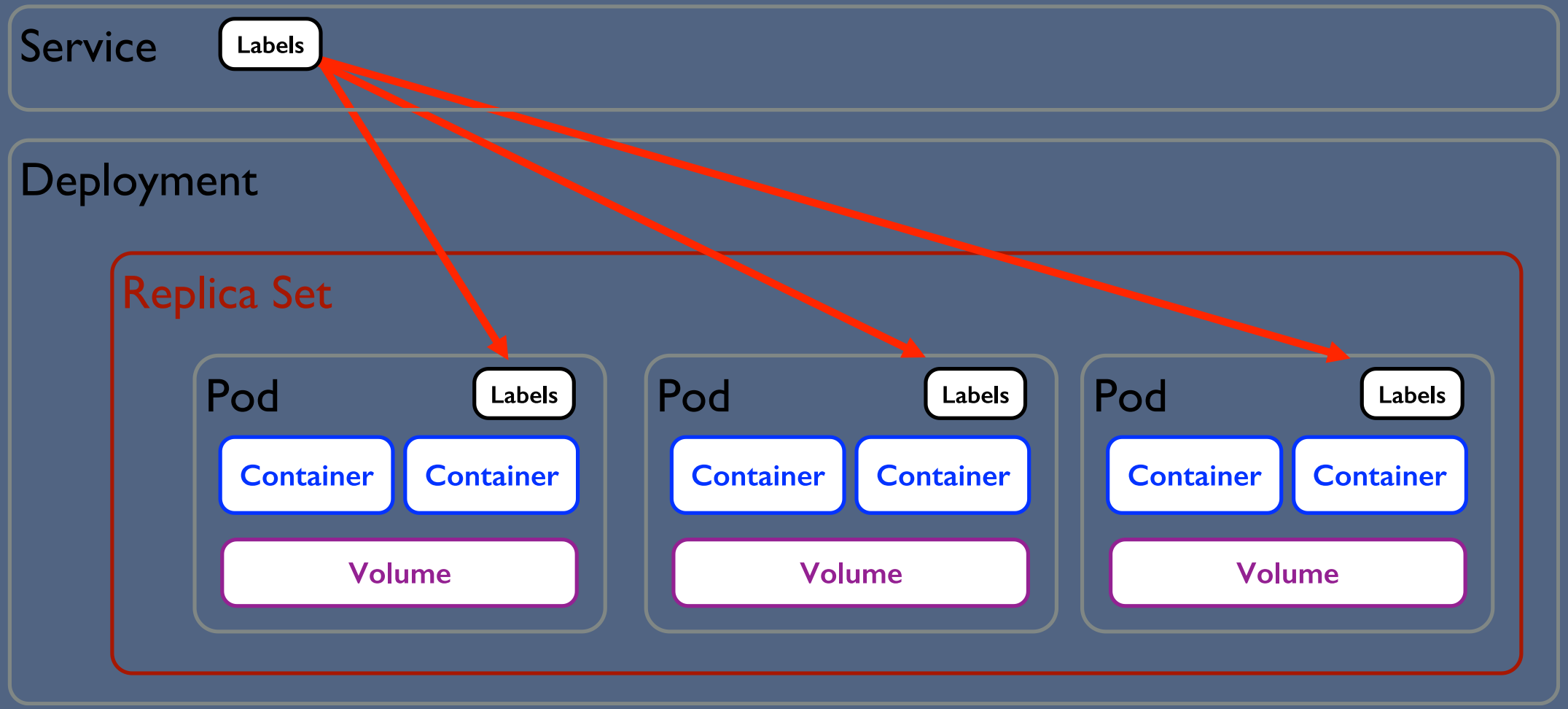
https://en.wikipedia.org/wiki/Kubernetes

Kubernetes Deployment Model

- Ingress
  - Exposes Service outside of Kubernetes
  - Maps URL paths to services

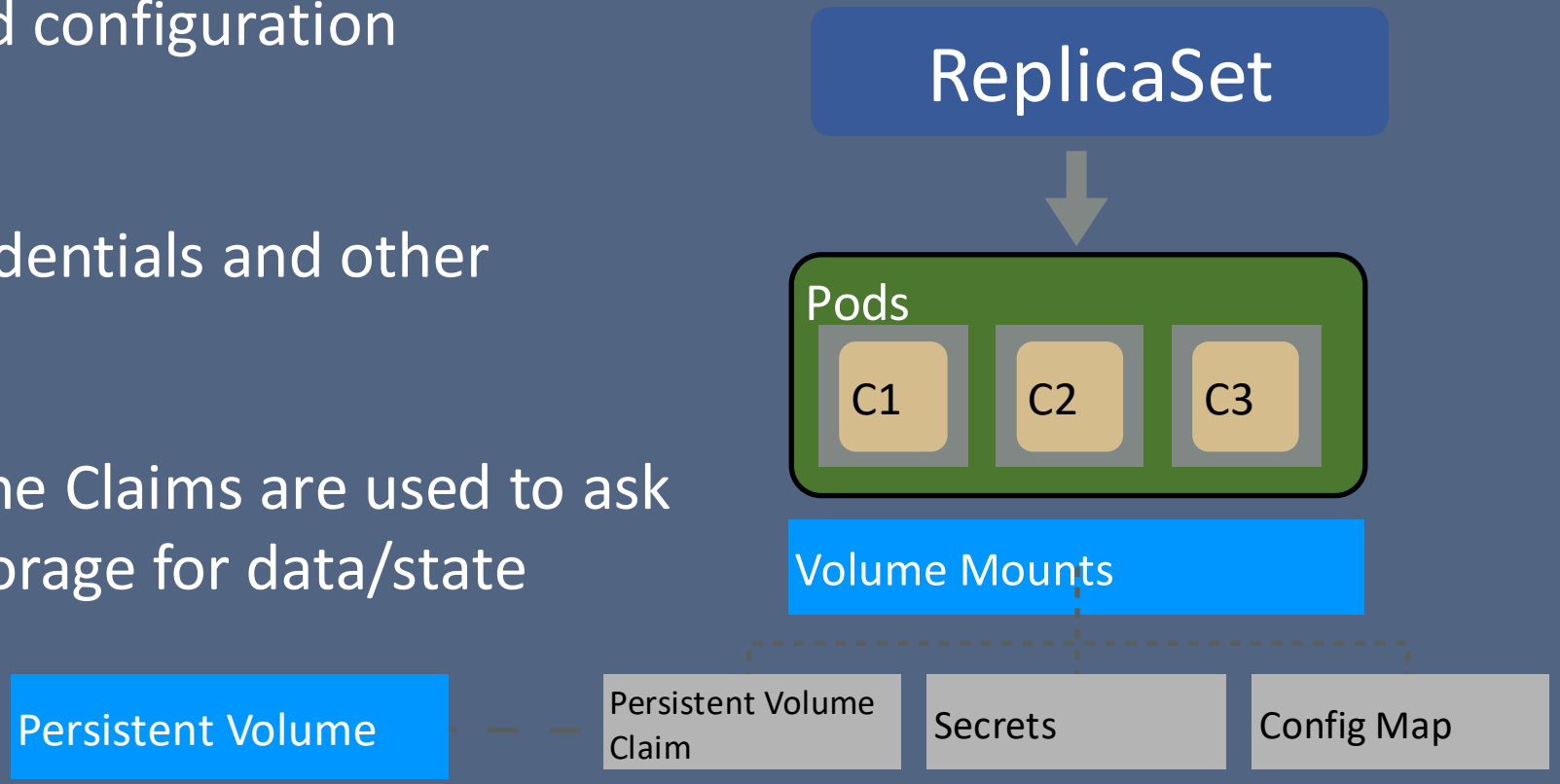


Kuberntes Logical View: Putting all together



Kubernetes Volume Mounts

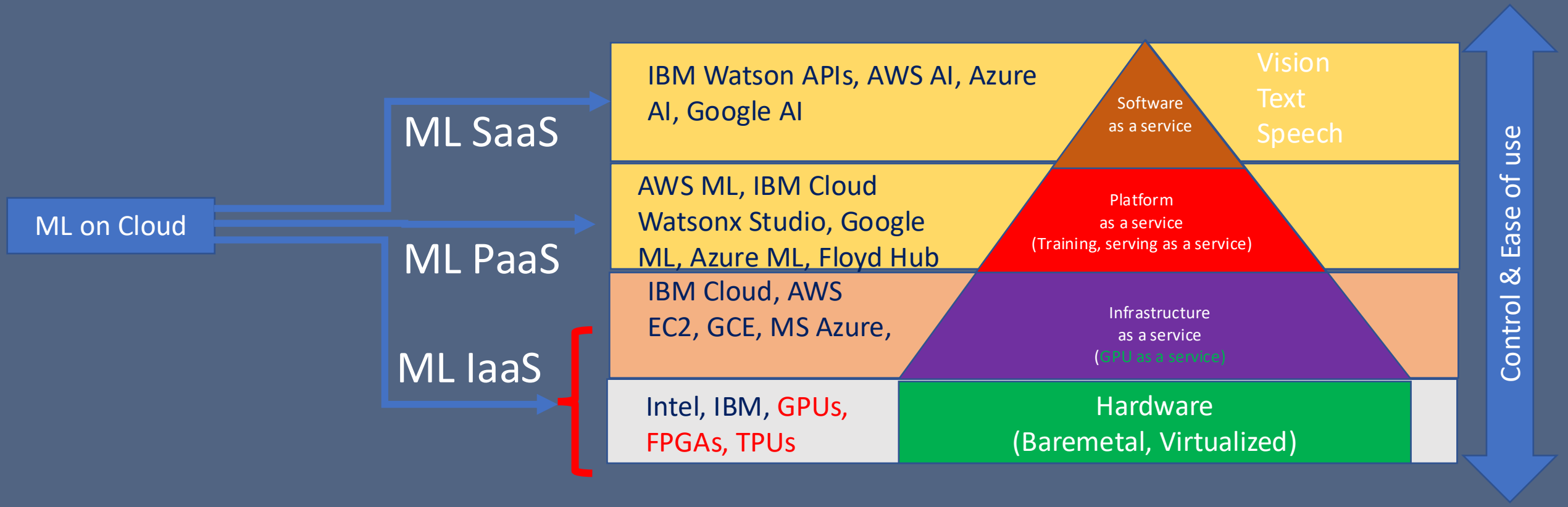
- Volumes
  - ConfigMaps hold configuration parameters
  - Secrets hold credentials and other secrets
  - Persistent Volume Claims are used to ask for persistent storage for data/state



Kubernetes machinery

- **Kube API Server**
- **Kube Scheduler**
- **Controller-manager**
- Kubelet
- Kube-proxy

Machine learning on cloud



Some market view

- According **Gartner**, GenAI Software will grow to $297B by 2027
- Generative AI is the leading area for massive growth
- High business value but
  - GenAI is still a complex,
  - Training models is too expensive
  - Applying DL in risky,
  - and time-consuming effort
  - to integrate, validate, and optimize deep learning solutions.

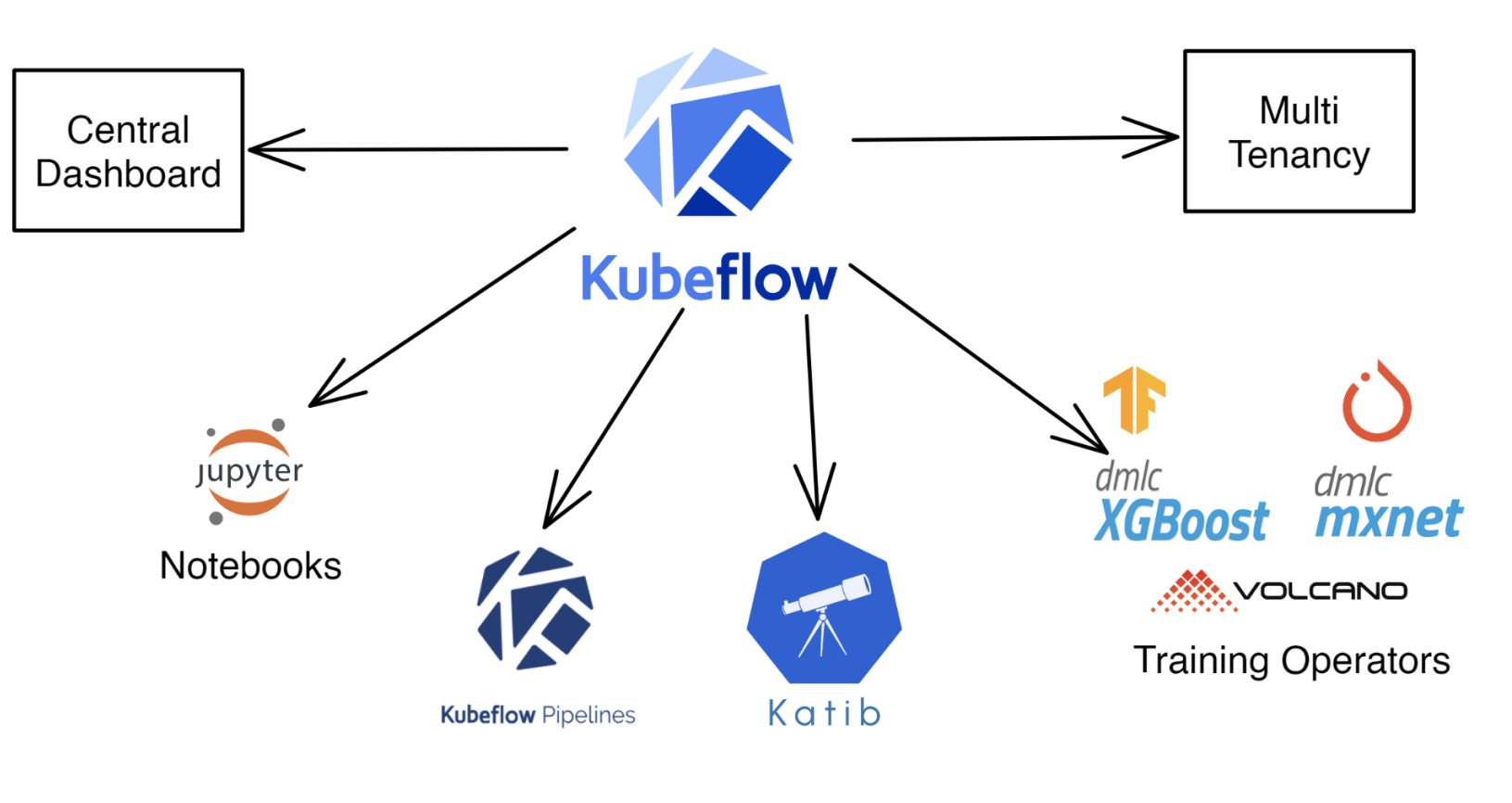
KubeFlow

- *Machine learning toolkit for Kubernetes*
  - https://www.kubeflow.org/docs/about/kubeflow/
- Goal:
  - Make scaling ML models, deploying them to production simple
- Uses Kubernetes
  - Easy, repeatable and portable deployment on diverse infrastructure
  - Deploy and manage loosely coupled micro-services
  - Scale based on demand
- Single tool that supports: Data preparation, model training, prediction serving, service management

What is KubeFlow?

- ML toolkit for Kubernetes
- Open source and community driven
- Supports multiple ML frameworks (Jupyter Notebook, Chainer, MxNet, PyTorch, Tensorflow, etc)
- Provides end to end workflows that can be shared, scaled and deployed

KubeFlow components



https://www.civo.com/docs/machine-learning/kubeflow-dashboard

KubeFlow components

Central Dashboard
The central user interface (UI) in Kubeflow

Kubeflow Notebooks
Documentation for Kubeflow Notebooks

Kubeflow Pipelines
Documentation for Kubeflow Pipelines.

Kubeflow Trainer
Documentation for Kubeflow Trainer

Katib
Documentation for Kubeflow Katib

Model Registry
Documentation for Kubeflow Model Registry

Spark Operator
Documentation for Spark Operator

Logical components that make up Kubeflow
https://www.kubeflow.org/docs/components/

Hands on session

What we will cover

- Create a kuberntes cluster in GCP
- Setup command line environment
- Install Kubeflow training operator
- Run a training workload

Installing Kubeflow Training operator

Install
kubectl apply -k "github.com/kubeflow/training-operator/manifests/overlays/standalone"

Create a job
kubectl create -f https://raw.githubusercontent.com/kubeflow/training-operator/master/examples/pytorch/simple.yaml

https://github.com/kubeflow/training-operator
https://www.kubeflow.org/docs/components/training/pytorch/#creating-a-pytorch-training-job

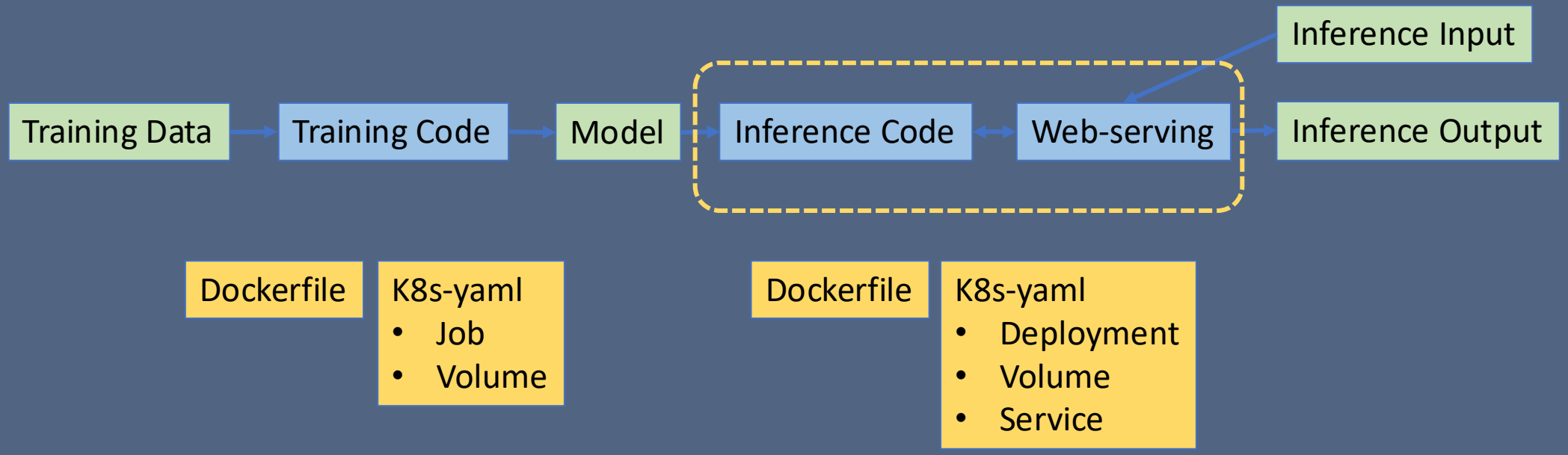
Homework #3 – Simple DL Workflow in Kubeflow

- Objective:
  - Develop Container and Kubernetes artifacts perform DL training and DL inference hosting in GKE: Google Kubernetes Engine.
  - The work demands self-driven deep dive into Kubernetes, Kubeflow concepts and components.
- Submission:
  - Dockerfile, K8S yaml files, instructions on how to build and run the service on K8S clusters on GCP-GKE. Screen captures on how the app run.
  - Document your work and report on your experiences: e.g.
    - What Kuberentes controllers did you use for training and inference and why?

Homework #3 – Architecture Aspect

- Application containers
  - Training: training program, container definition file (dockerfile), training job yaml file
  - Inference: inference program, dockerfile, service yaml file, deployment yaml file
    - Training and inference program need to store and load the trained model persistently.
    - Host a URL to allow interaction with User.
- User interaction
  - Give any input to the inference engine (a number, a space bar, image file, image id in the data set) to show that the inference is happening in an interactive/controlled manner.
- Persistent storage (PVC) – if required (optional)
  - After creating your cluster, you can create a PVC resource:
    - GCP → Kubernetes Engine → Storage → Persistent storage volume

The Abstracted Workflow



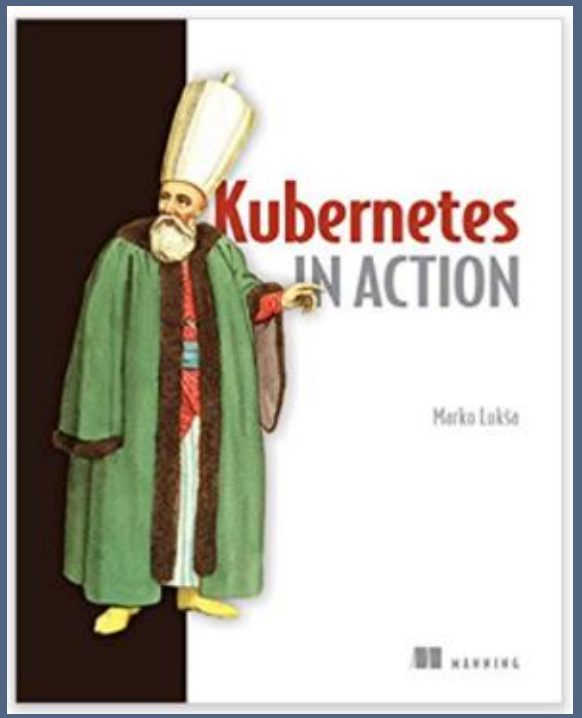
Rubric			Grading	
Category	**Partial**	**Deduction rules**	**partial score**	**Deduction comments**
Created k8s cluster	40	deduct 4 for not providing reasonable amount of proof of work.		
Training in k8s	30	deduct 2 for missing any of the steps: dockerfile, docker push, train-yaml, model saving		
Inferencing in k8s	25	deduct 2 for missing any of the steps: PVC creation and preparation, code augmentation, dockerfile, infer-yaml, infer-test		
writing	5	up to 2 point deduction if lack of attention to have readers to follow.		
late charge				
TOTAL SCORE	100			

Example on how to add a GPU to K8S Cluster

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: busybox
    command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 36']
    resources:
      limits:
        nvidia.com/gpu: 1
  nodeSelector:
    cloud.google.com/gke-accelerator: nvidia-tesla-t4
```

Suggested Study Material

- Borg, Omega, and Kubernetes:
  - https://ai.google/research/pubs/pub44843
- Introduction to Kubernetes:
  - https://www.digitalocean.com/community/tutorials/an-introduction-to-kubernetes
- Facebook Tupperware:
  - https://engineering.fb.com/data-center-engineering/tupperware/
- Kubernetes deconstructed:
  - https://www.youtube.com/watch?v=90kZRyPcRZw
  - http://kube-decon.carson-anderson.com/Layers/0-Intro.sozi.html#frame5378
- The History of Kubernetes on a Timeline:
  - https://blog.risingstack.com/the-history-of-kubernetes/
- The State of the Kubernetes Ecosystem [eBook]:
  - https://thenewstack.io/ebooks/kubernetes/state-of-kubernetes-ecosystem/
- Kubernetes In Action by Marko Luksa [Book]



Lab

- Try the various example files here:
  - https://github.com/seelam/k8s